1.3 Zaawansowany Prompt Engineering: Chain-of-Thought, System Prompts i techniki ograniczania halucynacji

Moduł 1: Fundamenty i Mechanizmy LLM

Krzysztof Rytlewski · 2026

Prompt Engineering — definicja i znaczenie

Prompt engineering to sztuka i nauka formułowania zapytań do modeli językowych w taki sposób, aby uzyskać odpowiedzi wysokiej jakości, spójne z oczekiwaniami i wolne od błędów. Łączy wiedzę o architekturze LLM, psychologię komunikacji i inżynierię oprogramowania.

Ten sam model, przy identycznych wagach, może dać diametralnie różne wyniki w zależności od sformułowania promptu. Dodanie frazy *„Let's think step by step"* zwiększyło dokładność modeli na zadaniach matematycznych o kilkadziesiąt procent — bez żadnej zmiany architektury.

Trzy warstwy promptu:

- **System prompt** — instrukcje definiujące rolę, styl i ograniczenia modelu
- **Few-shot examples** — przykłady par wejście–wyjście pokazujące pożądane zachowanie
- **User turn** — właściwe zapytanie użytkownika

System Prompts — definiowanie roli i granic modelu

System prompt to specjalna część kontekstu widoczna przez modelem przed pierwszą wiadomością użytkownika. Przekazywany jest jako wiadomość z rolą `system` w API OpenAI i Anthropic.

Cel system promptu:

- **Rola i persona** — np. „jesteś doświadczonym senior developerem Pythona"
- **Format odpowiedzi** — „zawsze odpowiadaj w formacie Markdown z nagłówkami H2"
- **Ograniczenia** — „nie odpowiadaj na pytania niezwiązane z programowaniem"
- **Ton i styl** — „mów bezpośrednio, unikaj zbędnych wstępów i podsumowań"
- **Kontekst domenowy** — firmowe zasady, pojęcia branżowe, struktura kodu

System prompt to kontrakt między deweloperem a modelem. Jego jakość bezpośrednio przekłada się na jakość i przewidywalność całej aplikacji.

Praktyczne wskazówki dla System Promptów

1. **Bądź konkretny, nie abstrakcyjny** — „Odpowiedź ma mieć maksymalnie 3 akapity" działa lepiej niż „Bądź zwięzły"

2. **Definiuj personę przez zachowanie** — zamiast „jesteś pomocny" napisz „zawsze sugerujesz następny krok działania"

3. **Testuj negatywne przypadki** — sprawdzaj, czy model respektuje ograniczenia przy próbach ich obejścia przez użytkownika

4. **Wersjonuj system prompty jak kod** — każda zmiana powinna być śledzona w repozytorium ze zrozumiałym opisem powodu modyfikacji

Chain-of-Thought — model myśli krok po kroku

Chain-of-Thought (CoT) to technika, w której model jest zachęcany do zapisania toku rozumowania prowadzącego do odpowiedzi, zamiast podania jej bezpośrednio. LLM rozwiązują trudniejsze zadania znacznie skuteczniej, gdy „myślą na głos".

Dlaczego CoT działa? Wygenerowane pośrednie kroki stają się częścią kontekstu następnych tokenów — pełnią rolę „roboczej pamięci" modelu i umożliwiają mu korzystanie z wyników własnych wcześniejszych obliczeń.

Wariant	Opis
Zero-shot CoT	Dodanie „Myśl krok po kroku" do promptu
Few-shot CoT	Przykłady z widocznym tokiem rozumowania
Self-consistency	Wiele łańcuchów + wybór najczęstszej odpowiedzi
Tree of Thought	Eksploracja drzewa możliwych ścieżek rozwiązania

Few-shot Prompting i Role Prompting

Few-shot prompting polega na dostarczeniu modelu kilku przykładów pary wejście–wyjście przed właściwym pytaniem. Uczy model oczekiwanego formatu, stylu i poziomu szczegółowości — bez żadnego dodatkowego trenowania.

Zasady skutecznego few-shot:

- Przykłady powinny być **reprezentatywne** dla typowych przypadków, nie skrajnych
- Kolejność przykładów ma znaczenie — modele przykładają większą wagę do tych **bliższych końcowi** (*recency bias*)
- Używaj **różnorodnych** przykładów, aby zapobiec nadmiernemu dopasowaniu do jednego wzorca

Role prompting — przypisanie modelowi konkretnej roli eksperckiej uruchamia specyficzny rejestr wiedzy i stylu. Model przewiduje tokeny pasujące do tej persony, bo w danych treningowych takie teksty cechowała wysoka jakość merytoryczna.

Halucynacje — źródła i mechanizmy powstawania

Halucynacje to przypadki, gdy model generuje treść brzmiącą wiarygodnie, ale faktycznie błędną — nieistniejące cytaty, zmyślone funkcje bibliotek, nieprawdziwe daty lub dane statystyczne.

Skąd biorą się halucynacje:

1. **Natura autoregresji** — model zawsze wybiera „najbardziej prawdopodobny" następny token, nawet gdy brakuje mu wiedzy na dany temat
2. **Brak reprezentacji niepewności** — baseline LLM nie potrafi powiedzieć „nie wiem"; tej umiejętności uczy dopiero RLHF
3. **Dziury w danych treningowych** — model interpoluje wzorce z podobnych, lepiej reprezentowanych tematów
4. **Pytania sugestywne** — fałszywa przesłanka zawarta w pytaniu jest często potwierdzana zamiast zakwestionowana

Technika Grounding — dostarczaj fakty w kontekście

Grounding polega na dostarczeniu modelu dokumentów jako części kontekstu i nakazaniu mu odpowiadania wyłącznie na ich podstawie — zamiast pytania o fakty z pamięci modelu.

```
Na podstawie poniższego dokumentu odpowiedz na pytanie.
Jeśli odpowiedzi nie ma w dokumencie, napisz "Nie znaleziono w źródle".

[DOKUMENT]
...
[KONIEC DOKUMENTU]

Pytanie: ...
```

Jest to fundamentalna technika systemów RAG (Retrieval-Augmented Generation) — model wie tylko tyle, ile mu dostarczysz w kontekście, co radykalnie zmniejsza ryzyko halucynacji.

Techniki ograniczania halucynacji — przegląd

- **Known-unknown prompting** — instruuj model: „Jeśli nie jesteś pewien, powiedz: »Zweryfikuj to w oficjalnej dokumentacji«"
- **Weryfikacja przez model (self-check)** — po uzyskaniu odpowiedzi wyślij osobne pytanie: „Czy Twoja odpowiedź zawiera twierdzenia, które mogą być nieprawdziwe?"
- **Niska temperatura (0.0–0.3)** — wymusza wybór przewidywalnych, bezpieczniejszych tokenów; stosuj w zastosowaniach faktograficznych
- **Strukturalny format JSON/XML** — ścisły format wymusza precyzję i utrudnia „przemycanie" halucynacji w płynnym tekście

Połączenie **groundingu**, **niskiej temperatury** i **wymuszonego formatu JSON** to najskuteczniejsza trójka technik redukcji halucynacji w systemach produkcyjnych.

Meta-prompting i Prompt Chaining

Meta-prompting — prosisz model o wygenerowanie lub ulepszenie promptu dla innego zadania. Model często lepiej zna własne ograniczenia niż deweloper i proponuje skuteczniejsze sformułowania.

Prompt chaining — podział złożonego zadania na sekwencję prostszych kroków, gdzie wyjście jednego promptu staje się wejściem kolejnego:

1. Wyodrębnij kluczowe wymagania z opisu zadania
2. Zaproponuj 3 architektury rozwiązania
3. Oceń każdą architekturę pod kątem skalowalności i kosztu
4. Wygeneruj kod dla najlepszej architektury

Każdy krok jest prostszy i mniej podatny na błędy. Łańcuchowanie promptów z pętlą sprzężenia zwrotnego to fundament działania agentów AI.

Kluczowe wnioski — sekcja 1.3

- Prompt engineering to dźwignia pozwalająca wyciągnąć z modelu znacznie więcej niż domyślne zachowanie sugeruje — bez zmian w architekturze
- System prompt to kontrakt z modelem; wersjonuj go jak kod i testuj negatywne przypadki
- CoT działa, bo pośrednie kroki rozumowania stają się częścią kontekstu i pozwalają modelowi „rozumować" nad własnymi wynikami
- Halucynacje można ograniczać przez grounding, niską temperaturę i wymuszone formaty — ale nie da się ich całkowicie wyeliminować samym promptingiem
- Prompt chaining i meta-prompting umożliwiają rozwiązywanie złożonych problemów przez dekompozycję na prostsze, zarządzalne kroki